

TD3

Exercice 1 :

On souhaite écrire une fonction **compteur()** qui :

- Initialise une variable locale $n = 0$,
- Définit une sous-fonction **incremente()** qui augmente n de 1 à chaque appel,
- Retourne la sous-fonction **incremente**.

Exécutez ensuite :

```
c1 = compteur()
c2 = compteur()
print(c1(), c1(), c2(), c1(), c2())
```

1. Quelle sera la sortie ? Pourquoi ?
2. Modifiez le code pour **rendre n accessible depuis l'extérieur** sans l'utiliser comme variable globale.

Exercice 2 :

On vous donne la liste suivante :

```
etudiants = [("Ali", 14), ("Sara", 18), ("Youssef", 11), ("Nada", 19), ("Khalid", 14)]
```

1. Triez cette liste selon les notes, puis par ordre alphabétique en cas d'égalité, **en une seule ligne**.
2. Filtrez la liste pour obtenir uniquement les étudiants ayant une note **supérieure à la moyenne** (utilisez filter).
3. Transformez les notes en mentions ("Passable", "Bien", "Très Bien") à l'aide de map et lambda.

Exercice 3 :

1. Écrire une fonction récursive **somme_chiffres(n)** qui retourne la somme des chiffres d'un entier positif n.

Ex. : `somme_chiffres(1234)` → 10

2. Définissez deux fonctions :

`def pair(n): ...`

`def impair(n): ...`

qui déterminent récursivement si un nombre est pair ou impair **sans utiliser l'opérateur %**.

Exercice 4 :

1. Créez une fonction **`appliquer(f, L)`** qui applique une fonction `f` à chaque élément d'une liste `L` (sans utiliser `map`).
2. Créez une fonction **`compose(f, g)`** qui retourne une **nouvelle fonction** équivalente à la composition `f(g(x))`.
3. Testez avec :

`def double(x): return 2 * x`

`def carre(x): return x * x`

`h = compose(carre, double)`

`print(h(5))` # Résultat attendu : 100

Exercice 5 :

1. Écrivez un décorateur `chrono` qui mesure le temps d'exécution d'une fonction.
2. Écrivez un décorateur `memo` qui **met en cache les résultats** d'une fonction récursive.
3. Appliquez les deux décorateurs à la fonction **`fibonacci(n)`** qui calcule le n^e terme de la suite de Fibonacci.
4. Essayez d'empiler les deux décorateurs et observez l'ordre d'exécution.

Exercice 6 :

1. Créez un générateur **`pairs(n)`** qui produit les nombres pairs jusqu'à `n`.
2. Créez un générateur **`entiers()`** qui génère une suite infinie d'entiers positifs.
3. Créez un autre générateur **`carres(gen)`** qui reçoit un générateur et produit le carré de chaque élément.
4. Affichez les 10 premiers carrés en combinant les deux via `islice` du module `itertools`.
5. Écrivez un décorateur **`journalise_gen`** qui affiche un message à chaque valeur produite par un générateur.

Exercice 7 :

Implémentez un mini système de traitement de données :

- Un générateur ***lecture_donnees()*** produit des nombres aléatoires entre 0 et 100.
- Une fonction ***filtre_pair()*** élimine les nombres impairs.
- Une fonction ***normalise()*** transforme les valeurs entre 0 et 1.
- Une fonction décorée ***affiche_etapes()*** affiche chaque transformation appliquée.
- Enchaînez le tout pour traiter les 10 premiers nombres du flux.